Yanasse: Finding New Proofs from Deep Vision's Analogies

Part 11: Summary of the Initial Campaign

100 Transfer Attempts, 10 Papers, 35 New Theorems

Alexandre Linhares*

April 2026

Abstract

This paper summarizes the initial Yanasse campaign: 100 schema transfer attempts across 10 (source, target) area pairs, reported individually in Parts 1–10. The system discovers new proofs by transferring tactic invocation patterns from one area of Lean 4's Mathlib library to a structurally distant area, using a domain-independent relational analogy engine (Deep Vision) to identify structurally similar proof states across areas, and an AI reasoning agent (Claude) to semantically adapt the source tactic's mathematical strategy to the target theorem.

Of 100 attempts, 35 produced Lean-verified new proofs (35% overall). The results reveal a two-factor model of transferability: success depends on (1) the source area's tactic profile (structural combinators transfer broadly; algebraic decision procedures transfer narrowly) and (2) the target area's type-class landscape (areas with rich algebraic instances are accessible to more sources). An element/morphism paradigm boundary separates areas where algebraic tactics gain purchase from those where they cannot. Information Theory is the most accessible target (90%); Condensed Mathematics the least (10%). Probability is the stronger source (46%); Number Theory the weaker (10%), but the two access non-overlapping parts of shared targets.

We distill lessons for improving the general theorem prover: which areas to prioritize as analogy sources, which QAP score ranges are predictive, which tactic classes transfer reliably, and where the matching engine's blind spots lie.

*alexandre@linhares.ltd. Affiliations: Getulio Vargas Foundation and ARGO LABS

Contents

1 Introduction **3**

2 Goals of the Experiment **3**

3 Summary of Results **4**
3.1 Cumulative results across all ten pairs . . . 4
3.2 Results by source area . . . 4
3.3 Results by target area . . . 4
3.4 Discussion of results . . . 5

4 What Transfers **5**
4.1 Structural combinators transfer broadly . . . 5
4.2 The (head, modifier) decomposition . . . 6
4.3 Algebraic tactics transfer narrowly but genuinely . . . 6
4.4 Meta-tactics transfer independently of domain content . . . 6
4.5 Automation substitution as a transfer mode . . . 7
4.6 Convergence points signal deep structural affinity . . . 7

5 What Does Not Transfer **7**
5.1 The type-class barrier . . . 7
5.2 The goal-shape barrier . . . 8
5.3 The homogeneity barrier . . . 8
5.4 Vacuous compilation: a false positive mode . . . 8
5.5 Degenerate attractors . . . 8

6 Improving the General Theorem Prover **8**
6.1 Which areas should the prover look to for analogues? . . . 8
6.2 Which areas should be de-prioritized? . . . 9
6.3 Which QAP scores are informative? . . . 9
6.4 How could QAP matching be improved for Lean/Mathlib? . . . 10
6.5 Which tactics should be prioritized? . . . 10
6.6 Which tactics should be de-prioritized? . . . 12

7 The Two-Factor Model **12**
7.1 Complementary sources . . . 12

8 The Structural Map of Mathlib Transferability **13**
8.1 Naive vs. weighted rankings . . . 13

9 Toward a Metatheoretic Impossibility Result **16**
9.1 The claim . . . 16
9.2 Three layers of formalization . . . 16
9.3 Why this matters . . . 17

10 Conclusion **17**

1 Introduction

Can a proof technique routine in one branch of mathematics be systematically discovered in a distant branch where it is unknown? The Yanasse project is a large-scale empirical test of this question, using relational analogy as the discovery mechanism.

The project takes its name from Horacio Hideki Yanasse, who taught that the best ideas cross disciplinary boundaries. The underlying engine—Deep Vision [11, 21]—grows out of the Fluid Analogies Research Group tradition [12–14], which holds that analogy-making—perceiving shared relational structure across superficially different situations—is the core of cognition. Deep Vision represents any structured domain as a *relational network*: entities connected by typed, directed relations. An analogy optimization finds the entity permutation that maximizes relational consistency between two networks. The same matching code that discovers chess analogies [15–20] discovers proof-state analogies—only the relation extractor is domain-specific.

The proof-state matcher has previously been used to discover a formal proof of Wolstenholme's theorem [22] in Lean 4 (see also [23]).

The Yanasse campaign applies this framework at scale. We extract tactic usage distributions across 27 top-level Mathlib namespaces (217,133 proof states), compute z-scores to identify tactics heavily used in a *source* area but rare or absent in a *target* area, match source and target proof states via GPU-accelerated analogy on Apple's MPS backend, and then ask an AI reasoning agent to *semantically adapt*—not symbol-substitute—the source tactic's invocation pattern to the target theorem. The adapted proof is compiled in Lean 4 with full Mathlib; exit code 0 with no `sorry` constitutes verification.

This summary paper synthesizes results from Parts 1 through 10 [1–10], covering 2 source areas (Probability and Number Theory), 8 target areas, 100 schema transfer attempts, and 35 new Lean-verified proofs.

2 Goals of the Experiment

The Yanasse campaign was designed to answer four questions:

1. **Can relational analogy discover cross-domain proof techniques?** The core hypothesis is that a domain-independent analogy engine—one that sees only the *shape* of proof states, not their mathematical content—can identify structural connections between distant areas of formalized mathematics that are sufficient to guide tactic transfer.

2. **What determines whether a tactic transfers?** Tactic transfer requires both a source pattern and a receptive target. We sought to identify the structural factors on each side that predict success or failure, beyond the trivial observation that "similar areas are easier."

3. **Are there systematic barriers to transfer?** If certain kinds of tactics *never* transfer to certain kinds of targets, understanding why would be as valuable as the successful transfers. We sought to map these barriers empirically.

4. **Can different source areas complement each other?** If two source areas access different layers of the same target, a multi-source campaign could achieve broader coverage than any single source. We tested this by sending both Probability and Number Theory to overlapping targets.

The campaign was calibrated against a pinned target (`nlinarith [sq_nonneg ...]` from Analysis to Combinatorics) before scaling. The calibration itself failed—the pinned schema was already present in the target area—but the pipeline was validated on an alternative pair, and the failure informed the schema-filtering rules used throughout the campaign.

3 Summary of Results

3.1 Cumulative results across all ten pairs

Table 1 presents the complete results. Across 100 schema transfer attempts on 10 area pairs, 35 produced Lean-verified new proofs.

Table 1: Cumulative results across all ten Yanasse pairs.

#	Pair	Schemas	New thm	Rate	Pool*
1	Prob → RepTheory	10	4	40%	672
2	Prob → SetTheory	10	4	40%	2,076
3	Prob → Condensed	10	2	20%	189
4	Prob → FieldTheory	10	4	40%	2,792
5	Prob → Dynamics	10	4	40%	495
6	Prob → ModelTheory	10	5	50%	1,079
7	Prob → InfoTheory	10	9	90%	240
8	NumTh → Condensed	10	0	0%	189
9	NumTh → RepTheory	10	0	0%	672
10	NumTh → Dynamics	10	3	30%	495
	Total	**100**	**35**	**35%**	—

*Pool = number of proof states in the target area's Mathlib corpus, against which each source proof is matched.

3.2 Results by source area

Table 2: Transfer rates by source area.

Source	Pairs	Schemas	Successes	Rate
Probability	7	70	32	46%
Number Theory	3	30	3	10%
Total	**10**	**100**	**35**	**35%**

Probability's 46% rate reflects the domain-agnostic character of its distinctive tactics: structural combinators (`filter_upwards`, `ext1`, `congr with`, `any_goals`, `by_cases`) that operate on logical structure rather than algebraic content. Number Theory's 10% rate reflects the opposite: its distinctive tactics (`ring`, `decide`, `norm_num`, `linarith`, `field`) are algebraic decision procedures that require specific type-class instances to execute.

3.3 Results by target area

The target difficulty spectrum ranges from 90% (Information Theory) to 10% (Condensed Mathematics). Notably, targets tested with both source areas (Dynamics, Representation Theory, Condensed)

Table 3: Transfer rates by target area (all sources combined).

Target area	Attempts	Successes	Rate
Information Theory	10	9	90%
Model Theory	10	5	50%
Field Theory	10	4	40%
Set Theory	10	4	40%
Dynamics	20	7	35%
Representation Theory	20	4	20%
Condensed	20	2	10%

show that adding Number Theory as a second source adds new theorems only for Dynamics—the one target among the three that contains element-level arithmetic sub-layers.

3.4 Discussion of results

The 35% overall rate means that roughly one in three attempts at transferring a proof technique across Mathlib area boundaries produces a Lean-verified new proof. This is substantially above what a random-guess baseline would yield: the schemas were selected precisely because they are *rare or absent* in the target area, so the prior probability that they would compile at all is low.

The variance across pairs is large (0%–90%), and this variance is itself the most informative result of the campaign. It reveals that transfer success is not a property of the method alone—it depends on a structured interaction between source tactic profiles and target type-class landscapes, as we analyze in the following sections.

4 What Transfers

Across the 35 successful transfers, several patterns emerge.

4.1 Structural combinators transfer broadly

The most reliable transfers come from Probability's *structural combinators*—tactics whose function is to reorganize proof state rather than to compute with algebraic content:

- `ext1` (function/morphism extensionality): succeeded in 5 of 7 target areas (RepTheory, SetTheory, Condensed, Dynamics, InfoTheory). Extensionality is mathematically universal: every area has objects whose equality reduces to pointwise equality.
- `by_cases` (case splitting on a structural property): succeeded in 5 of 7 target areas (RepTheory, SetTheory, FieldTheory, ModelTheory, InfoTheory). Every area has natural dichotomies (zero/nonzero, empty/nonempty, finite/infinite).
- `any_goals` (dispatch any trivially closeable sub-goal): succeeded in 5 of 7 target areas (RepTheory, SetTheory, FieldTheory, ModelTheory, InfoTheory). After a case-split or decomposition, at least one branch is often trivial.
- `congr with` / `congrm` (congruence with named variables): succeeded in 3–4 targets. Peeling one structural layer and naming the bound variable is a domain-independent operation.

- `push` (`push_neg`, `push_cast`, membership normalization): succeeded in 3 of 7 targets. Negation pushing and membership normalization operate on logical connectives, not domain-specific content.

Finding 1 (Structural combinators as universal transfer vectors)**.** Probability's structural combinators—`ext1`, `by_cases`, `any_goals`, `congr with`, `push`—transfer to 5 or more of 7 target areas. Their universality comes from operating on logical structure (quantifiers, disjunctions, equality, membership) rather than algebraic content. These are the most reliable class of tactics for cross-area transfer.

4.2 The (head, modifier) decomposition

Part 1 identified that tactic schemas decompose into a *head* (the tactic name) and a *modifier* (the argument structure: `with`-clause, bracketed lemma list, arity pattern). Across the campaign, this decomposition proved to be the single most predictive structural feature:

- The **head** is typically *domain-gated*: its elaborator requires domain-specific goal shapes or type-class instances. `filter_upwards` requires `Filter.Eventually`; `ring` requires `CommSemiring`; `measurability` requires σ-algebra structure.
- The **modifier** often expresses a *domain-general proof pattern*: "introduce a fresh element and name it," "coordinate over a list of facts," "dispatch any trivial sub-goal." These patterns have analogs across areas.

When the head fails but the modifier carries independent content, semantic adaptation can replace the head with a target-specific tactic while preserving the modifier's proof pattern. This is the mechanism behind the most interesting transfers: `filter_upwards [LIST] with` ω becomes `ext1` ω `+ simp [LIST]` in Representation Theory (Part 1); the same pattern recurs in Set Theory, Model Theory, and Information Theory.

4.3 Algebraic tactics transfer narrowly but genuinely

Number Theory's algebraic decision procedures—`ring`, `ring_nf`, `linarith`, `norm_num`—succeeded only 3 times in 30 attempts, all against Dynamics. But when they succeed, the transfer is genuinely mathematical: `ring` closes an algebraic identity in the translation-number computation for negative integer powers (Part 10, item 90); `ring_nf` normalizes an arithmetic sub-step in a semiconjugacy proof (Part 10, item 92). These are not structural repackaging—they are computational steps that the target proofs actually need.

4.4 Meta-tactics transfer independently of domain content

Part 10 revealed that `enter`—a tactic for navigating into nested expressions—succeeds with a QAP score of 0.0. Its utility depends on expression *nesting depth*, not on mathematical content. The relational analogy matcher, which compares entity-relation networks, cannot detect this kind of technique-level similarity. `enter` transferred to Dynamics (Part 10) despite zero structural similarity between the Number Theory and Dynamics proof states, because both domains contain expressions nested under scalar multiplication or function application where a surgical inner rewrite is needed.

4.5 Automation substitution as a transfer mode

Several successful transfers are *horizontal substitutions*: one automation tactic replaces another that accomplishes the same mathematical step. `measurability` replaces `fun_prop` in Information Theory (Part 7, item 65); `all_goals` replaces manual per-goal dispatch in Field Theory (Part 4) and Model Theory (Part 6). These transfers are less novel than genuine technique transfers, but they are real: the adapted proofs compile and close goals that the original automation did not reach in the target area.

4.6 Convergence points signal deep structural affinity

When multiple schemas independently route to the same target theorem via the QAP matcher, the cause matters. In Information Theory (Part 7), five schemas converge on `integral_llr_compProd_eq_add`—the chain rule for log-likelihood ratios—producing five distinct new proofs of the same theorem. Each proof articulates the same mathematical step differently (`congr with`, `ext1`, `measurability`, `congrm`, `split_ifs`). This *convergence point* signals that the source and target share deep mathematical infrastructure, and the QAP matcher correctly identifies it.

In contrast, the *degenerate attractor* phenomenon (Part 6: six schemas converge on `instSubsingleton`, a trivial 3-entity target) signals pool scarcity, not structural affinity. Distinguishing genuine convergence from degenerate attraction is diagnostically important: entity count and schema diversity are the key discriminators.

5 What Does Not Transfer

The 65 failures are at least as informative as the 35 successes. They cluster into three systematic barrier types.

5.1 The type-class barrier

The most common failure mode: the tactic requires a type-class instance (`CommSemiring`, `Field`, `LinearOrderedCommSemiring`, `Decidable`, `CanLift`) that the target's proof goals do not carry. This barrier operates at Lean 4's type-class synthesis level—before any mathematical reasoning begins, the tactic's elaborator fails to find the required instance.

This barrier is absolute when the target area operates entirely at the morphism level. In Condensed Mathematics (Part 8), proof goals involve functors, natural transformations, and adjunctions—types carrying no algebraic structure. In Representation Theory (Part 9), proof goals involve module homomorphisms and chain-complex morphisms—types carrying additive group structure but not ring structure. All 20 Number Theory attempts against these two targets failed for this reason.

Finding 2 (The element/morphism paradigm boundary)**.** There exists a paradigm boundary in Mathlib between areas whose proofs operate on *elements* of algebraic structures (rings, fields, ordered semirings) and areas whose proofs operate on *morphisms* between categorical objects (functors, natural transformations, module homomorphisms). Algebraic decision procedures can cross this boundary only where the target contains element-level sub-layers—as in Dynamics' translation numbers and Birkhoff averages (Part 10), which embed $\mathbb{R}$-arithmetic within otherwise topological proofs.

5.2 The goal-shape barrier

Some tactics require a specific goal shape that the target area simply does not generate. `fun_prop` requires goals tagged with `@[fun_prop]` attributes (`Continuous`, `Measurable`, `Differentiable`). `filter_upwards` requires `Filter.Eventually` or filter membership goals. `measurability` requires σ-algebra predicates. When the target area has zero such goals, no amount of semantic adaptation can bridge the gap.

This barrier differs from the type-class barrier in that it is about the *structure of the goal proposition* rather than the *algebraic structure of the types involved*. `fun_prop` failed in 5 of 7 Probability targets and in all 3 Number Theory targets—making it the least transferable tactic in the campaign.

5.3 The homogeneity barrier

`all_goals` requires every sub-goal after a decomposition to be closeable by a single decision procedure. In areas where decomposition produces *heterogeneous* sub-goals (a homology-class equality alongside a cycle-membership proof, or a measurability side-condition alongside an arithmetic identity), `all_goals` fails even though the individual sub-goals are tractable. Its sister tactic `any_goals`, which tolerates heterogeneity by succeeding if *any* sub-goal closes, transfers far more broadly.

5.4 Vacuous compilation: a false positive mode

Parts 8–9 identified a subtle failure: tactics that *compile* but whose distinctive component does zero work. `exact_mod_cast` and `rw_mod_cast` compiled in Representation Theory (Part 9, items 88–89) because the `mod_cast` preprocessor found no coercion lemmas to apply—so it silently passed through, and the plain `exact` or `rw` did all the work. These are *not* successful transfers: the distinctive component (cast normalization) contributed nothing. The campaign criteria correctly classify them as failures.

5.5 Degenerate attractors

Small target pools produce degenerate attractors: a few proof states with high entity counts that score well against all source queries, regardless of mathematical content. In Model Theory (Part 6), the `instSubsingleton` proof (3 entities) attracted 6 of 10 schemas. In Condensed Mathematics (Part 8), three target theorems attracted all 10 schemas. Degenerate attractors inflate QAP scores without reflecting genuine structural affinity, and the resulting transfers often succeed trivially or fail uninformatively.

6 Improving the General Theorem Prover

The Yanasse campaign was a stress test of the Deep Vision matching engine applied to formal mathematics. Its findings directly inform how the general theorem prover (`proving_loop.py` [21]) should prioritize analogy sources, interpret QAP scores, and select tactics.

6.1 Which areas should the prover look to for analogues?

Finding 3 (Source area prioritization)**.** When the prover encounters a proof state, it should prioritize analogy matches from source areas whose tactic profiles complement the target's needs:

1. **Probability** is the strongest general-purpose source (46% across 7 targets). Its structural combinators transfer broadly and operate on logical structure that every area shares. Probability should be the *default* high-priority source for any target area.

2. **Number Theory** is a specialist source (10% overall, but 30% against Dynamics). It should be prioritized *only* when the target proof state carries algebraic type-class instances (`CommSemiring`, `Field`, `LinearOrderedCommSemiring`). A type-class pre-check can filter this efficiently.

3. **Analysis** (untested but predicted): the campaign's two-factor model predicts that Analysis—with its mix of structural combinators (`filter_upwards`, `continuity`) and computational tactics (`norm_num`, `nlinarith`)—would be a strong general-purpose source, possibly rivaling Probability. It should be the next area tested.

4. **Algebra** (untested but predicted): with its rich decision procedures (`ring`, `group`, `field_simp`), Algebra would likely mirror Number Theory's profile: narrow but genuine transfers to targets with algebraic structure.

6.2 Which areas should be de-prioritized?

Finding 4 (Target area difficulty)**.** The prover should adjust its confidence in analogy-guided tactics based on the target area's position on the difficulty spectrum:

1. **Information Theory, Model Theory** ($\geq$50%): analogy matching is highly effective; the prover should trust analogy-suggested tactics and invest reasoning budget in adaptation.

2. **Set Theory, Field Theory, Dynamics** (35–40%): analogy matching works but is not dominant; the prover should use analogy suggestions as *first attempts* but fall back to standard tactics quickly.

3. **Representation Theory** (20%): analogy works only from structural sources (Probability); algebraic sources (Number Theory) are completely ineffective. The prover should restrict analogy sources to structural-combinator-heavy areas.

4. **Condensed Mathematics** (10%): analogy matching is nearly useless. The prover should de-prioritize analogy-guided tactics and invest in category-theory-specific automation (`aesop_cat`, categorical `simp` lemma sets) instead.

5. **Category Theory as source**: should be *excluded* from analogy source pools. Its DSL-style tactics (`cat_disch`, `aesop_cat`) encode domain-specific diagrammatic reasoning, not transferable proof strategies. They dominated the initial rankings and had to be filtered out.

6.3 Which QAP scores are informative?

The QAP (Quadratic Assignment Problem) matching score is the primary signal the prover uses to rank analogy candidates. The campaign reveals that its predictive value is real but bounded.

Finding 5 (QAP score interpretation)**.** 1. **Moderate scores (8–20) are the most predictive range.** The majority of successful transfers fall in this range. Items 90 (QAP=14.49, `ring`→Dynamics) and 92 (QAP=12.17, `ring_nf`→Dynamics) are representative: the matcher identified genuine structural similarity between proof states involving real-valued functions with algebraic equality goals.

2. **Very high scores (>100) often signal degenerate attractors or pool artifacts,** not genuine structural affinity. Items 64 (QAP=125.0) and 66 (QAP=118.75) in Information Theory succeeded, but items with similarly high scores in Model Theory converged on a trivial 3-entity target. The prover should treat scores above 100 with caution and cross-check against entity count.

3. **Zero scores can still yield successful transfers** for meta-tactics. Item 93 (`enter`, QAP=0.0) succeeded because the tactic's utility depends on expression nesting depth, not on the entity-relation structure that QAP measures. The prover should not discard zero-QAP candidates when the schema is a known meta-tactic.

4. **QAP is blind to type-class compatibility.** A high QAP score does not guarantee that the source tactic's type-class requirements are met in the target. Items 70 (QAP=17.75, `ring`→Condensed) and 80 (QAP=17.19, `ring`→RepTheory) scored well but failed completely. *The prover should add a type-class pre-filter before investing reasoning budget in adaptation.*

6.4 How could QAP matching be improved for Lean/Mathlib?

The campaign exposes four specific limitations of the current matching engine that, if addressed, would improve the prover's performance on Lean proof states:

1. **Type-class-aware scoring.** The 14 relation types extracted from proof states (rewrite, fit/apply, head-match, structure, etc.) capture *logical* relationships between entities but not *algebraic* relationships between types. Adding a relation type that encodes type-class instances (e.g., "this type carries `CommSemiring`") would allow the matcher to distinguish proof states where algebraic tactics can execute from those where they cannot—directly addressing the paradigm boundary.

2. **Entity-count normalization.** The current QAP score is unnormalized: larger proof states (more entities, more relations) produce higher raw scores. This creates a bias toward matching large source states with large target states, and causes small but structurally rich targets to be overlooked. Normalizing by $\sqrt{n_1 \cdot n_2}$ (geometric mean of entity counts) would make scores comparable across different-sized proof states.

3. **Nesting-depth features for meta-tactics.** Meta-tactics like `enter`, `conv`, and `show` operate on expression *shape* rather than mathematical *content*. The current relation extractor does not capture nesting depth, application depth, or binder structure. Adding expression-shape features—even simple ones like "maximum binder depth" or "number of nested function applications"—would let the matcher predict meta-tactic applicability.

4. **Area-aware weighting.** The campaign demonstrates that the same relation type (e.g., head-match) has different predictive value depending on the source-target pair. Head-match is highly informative for Probability→Information Theory (where shared measure-theoretic heads like `MeasureTheory.Measure` signal genuine affinity) but uninformative for Number Theory→Condensed (where the heads are categorically different). Learning area-specific relation weights from the campaign's 100 data points could improve matching precision.

6.5 Which tactics should be prioritized?

Finding 6 (Tactic prioritization for the prover)**.** Based on the 100-attempt campaign, the prover should adjust its tactic exploration order as follows:

High priority (transfer broadly, succeed reliably):

- `ext1`, `by_cases`, `any_goals`: succeeded in 5+ of 7 Probability targets. These are the workhorses of cross-area transfer.
- `congr with`, `push`: succeeded in 3–4 targets. Reliable when the target has equality or membership goals.
- `ring`, `ring_nf`: succeeded only against targets with `CommSemiring`, but when applicable, they close goals that no other tactic can. Should be tried whenever the type-class pre-filter passes.

Medium priority (succeed in specific contexts):

- `all_goals`: succeeded in 3 targets (Dynamics, FieldTheory, InfoTheory) but fails on heterogeneous sub-goal sets. The prover should check sub-goal homogeneity before trying.
- `measurability`: succeeded only where σ-algebra structure exists (InfoTheory horizontal substitution for `fun_prop`).
- `enter`: domain-independent but requires expression nesting. Check nesting depth, not QAP score.
- `split_ifs`: succeeded once (InfoTheory) on a piecewise-defined function. Should be tried when the target contains `if-then-else` expressions.

Low priority (rarely transfer):

- `fun_prop`: 1 success out of 10 attempts (only where measurability goals were already present). Domain-gated and highly specialized.
- `lift`: 0 successes in 10 attempts. Requires `CanLift` instances that are essentially unique to Probability's type ecosystem (`ENNReal`→`NNReal`, `WithTop`→base type).
- `congrm`: 1 success (InfoTheory). Requires matched outer-operator patterns between goal sides, which is rare outside measure theory.
- `decide`: 0 successes. Requires ground propositions with no free variables—extremely rare in abstract mathematics.
- `exact_mod_cast`, `rw_mod_cast`: 0 genuine successes (compiled twice but the cast component did zero work). Require numeric coercion chains absent outside Number Theory and Analysis.
- `norm_num`: 0 successes. Requires numeric literal content in goal.
- `linarith`: 0 successes outside its home areas. Requires linear-ordered commutative semiring, which is absent from most target types.
- `field`: 0 successes. Requires `Field` instance, absent from all tested targets.

6.6 Which tactics should be de-prioritized?

The campaign identifies a clear hierarchy. Tactics that *never* succeeded in cross-area transfer—`lift`, `decide`, `norm_num`, `linarith`, `field`, `exact_mod_cast`, `rw_mod_cast`—should be de-prioritized in the analogy-guided pipeline. This does not mean they are useless: `linarith` and `norm_num` are powerful *within* their home areas. But the analogy engine's job is to find *cross-area* connections, and these tactics are too domain-specific to serve as transfer vehicles.

The prover already includes these tactics in its standard-tactics fallback (step 7 of the pipeline). The campaign confirms that this is the right placement: they should be tried after domain-independent strategies are exhausted, not promoted by analogy matching.

7 The Two-Factor Model

The campaign's results are well described by a two-factor model:

$$P(\text{transfer success}) \;\approx\; f(\text{source tactic profile}) \;\times\; g(\text{target type-class landscape})$$

where:

- f measures how *domain-general* the source tactic is. Structural combinators ($f \approx 0.6$–0.7) transfer broadly; algebraic decision procedures ($f \approx 0.1$) transfer narrowly.
- g measures how *type-class-accessible* the target is. Information Theory ($g \approx 0.9$) shares Probability's measure-theoretic infrastructure; Condensed Mathematics ($g \approx 0.1$) has an entirely categorical type-class landscape.

The model explains the key patterns:

1. **Probability→InfoTheory (90%):** High f (structural combinators) $\times$ high g (shared measure-theoretic types) = very high transfer rate.
2. **NumTh→Condensed (0%):** Low f (algebraic procedures) $\times$ low g (no algebraic instances) = complete failure.
3. **NumTh→Dynamics (30%):** Low f $\times$ moderate g (partial $\mathbb{R}$-arithmetic sub-layers) = partial success, concentrated in specific sub-areas.
4. **Prob→Condensed (20%):** High f $\times$ low g = limited success, only from the most domain-agnostic tactics (`ext1`, `congr with`).

The model also predicts untested pairs. For instance, Probability→Analysis should achieve a very high rate (structural combinators + shared measure-theoretic types + Analysis has rich algebraic structure). Number Theory→Analysis should achieve a moderate rate (algebraic procedures + Analysis has `CommSemiring` everywhere).

7.1 Complementary sources

The two-factor model implies that different sources access different parts of the same target. This was confirmed empirically for Dynamics: Probability transferred structural combinators to the measure-theoretic and extensionality layers (4 new theorems); Number Theory transferred algebraic

procedures to the arithmetic core (3 new theorems). The two sets of new theorems are entirely non-overlapping.

A multi-source campaign is therefore strictly more powerful than any single-source campaign. The prover should maintain analogy libraries from multiple source areas and query them in parallel, weighted by the two-factor model's prediction for the current target.

8 The Structural Map of Mathlib Transferability

The campaign's data, combined with the transferability-weighted model of Section 7, allows us to compute the full transfer potential between every pair of Mathlib's 23 mathematical areas (excluding `CategoryTheory` as source and `Tactic/Control/Logic` as targets; see Part 1 [1] for filtering rationale). We present two matrices: the *naive* model (pair potential computed from raw z-score gaps, used to select the initial campaign's 10 pairs) and the *transferability-weighted* model (pair potential weighted by empirical transfer rates per tactic class, derived from the campaign's 100 data points).

Table 4 provides the area index used in both matrices. Tables 5 and 6 present the 23×23 transfer potential matrices. Each cell shows the pair potential (sum of top-10 schema transfer gaps, weighted by $\log(1 + \text{source count})$ and, in the weighted model, multiplied by the tactic class's empirical transfer probability). Higher values indicate greater theoretical transfer opportunity. The diagonal is excluded (self-transfer is trivial).

8.1 Naive vs. weighted rankings

The reranking is dramatic. In the naive model, Number Theory pairs dominate the top 20 (high z-score gaps from algebraic decision procedures). In the weighted model, those pairs drop sharply because the campaign showed algebraic procedures transfer at $\sim$10% vs. $\sim$60% for structural combinators. Probability remains the top source in both models, but the weighted model promotes new sources—Measure Theory, Algebraic Topology, Representation Theory—whose structural combinators were buried under Number Theory's inflated gaps.

The full matrices and the underlying data (`area_pair_potential.parquet`, `transfer_candidates.parquet`, `schemas.parquet`) are available in the project's GitHub repository.

Table 4: Area index for the transfer potential matrices.

#	Area	Mathlib namespace
1	Algebra	`Mathlib.Algebra`
2	Algebraic Geometry	`Mathlib.AlgebraicGeometry`
3	Algebraic Topology	`Mathlib.AlgebraicTopology`
4	Analysis	`Mathlib.Analysis`
5	Combinatorics	`Mathlib.Combinatorics`
6	Computability	`Mathlib.Computability`
7	Condensed	`Mathlib.Condensed`
8	Data	`Mathlib.Data`
9	Dynamics	`Mathlib.Dynamics`
10	Field Theory	`Mathlib.FieldTheory`
11	Geometry	`Mathlib.Geometry`
12	Group Theory	`Mathlib.GroupTheory`
13	Information Theory	`Mathlib.InformationTheory`
14	Linear Algebra	`Mathlib.LinearAlgebra`
15	Measure Theory	`Mathlib.MeasureTheory`
16	Model Theory	`Mathlib.ModelTheory`
17	Number Theory	`Mathlib.NumberTheory`
18	Order	`Mathlib.Order`
19	Probability	`Mathlib.Probability`
20	Representation Theory	`Mathlib.RepresentationTheory`
21	Ring Theory	`Mathlib.RingTheory`
22	Set Theory	`Mathlib.SetTheory`
23	Topology	`Mathlib.Topology`

Table 5: Naive transfer potential matrix (raw z-score gaps, no transferability weighting). Row = source area, column = target area. The initial campaign selected the 10 highest-potential pairs from this matrix.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1	–	33	33	23	33	33	49	25	49	49	49	49	49	23	46	49	23	23	33	49	7	49	11
2	74	–	152	86	134	166	175	119	162	122	99	142	175	121	142	170	98	140	150	167	56	151	118
3	34	70	–	7	61	103	141	38	149	82	42	90	137	31	84	116	42	58	96	79	31	107	33
4	79	120	120	–	74	114	129	118	129	116	80	106	129	118	84	129	60	114	99	129	74	129	28
5	49	92	76	43	–	89	127	45	116	98	97	72	127	82	83	115	81	68	92	127	82	104	72
6	47	135	138	59	90	–	165	58	143	134	116	98	152	111	89	143	77	118	127	154	73	143	60
7	8	26	43	13	20	64	–	23	64	20	47	20	101	8	21	55	28	20	44	71	8	55	
8	55	120	138	78	65	86	165	–	145	90	118	91	165	84	75	129	93	75	119	145	50	93	87
9	28	62	74	5	47	82	89	51	–	55	68	60	87	56	21	83	35	42	60	90	14	60	26
10	41	81	120	41	82	92	163	78	137	–	77	75	140	49	75	110	46	98	110	118	24	100	62
11	59	86	83	11	84	91	117	64	91	86	–	82	117	48	66	101	45	75	83	113	65	87	55
12	33	80	91	61	64	87	91	54	91	88	67	–	91	69	81	89	66	85	84	91	36	85	61
13	19	23	46		23	36	61	41	54	48	3	36	–	41	19	56		65	15	68	23	52	20
14	20	90	107	53	80	96	107	25	107	78	62	57	107	–	68	107	60	102	85	107	31	107	55
15	102	137	147	7	88	70	147	147	147	147	51	147	125	110	–	147	76	64	27	147	68	137	66
16	25	52	76	25	32	64	82	29	82	55	52	57	82	35	29	–	41	60	61	82	20	64	29
17	76	128	144	33	112	135	189	85	189	147	121	123	173	99	70	183	–	177	110	189	48	176	92
18	29	84	84	34	60	81	94	41	94	67	67	60	94	58	59	90	60	–	76	94	48	84	39
19	70	132	187	3	116	170	208	144	204	207	129	171	190	162	31	204	86	137	–	219	101	209	72
20	17	37	59	9	27	77	129	43	105	49	24	40	133	16	20	100	28	31	79	–	6	106	30
21	18	55	78	12	49	78	93	22	93	60	32	39	93	40	56	78	25	70	70	93	–	70	27
22	35	82	117	38	72	101	149	17	113	100	82	82	128	81	78	132	79	66	108	160	56	–	46
23	52	104	145	27	96	139	146	72	146	104	92	110	146	92	75	104	102	87	114	146	82	132	–

Table 6: Transferability-weighted potential matrix. Same structure as Table 5, but each schema's gap is multiplied by its tactic class's empirical transfer probability (from the campaign's 100 data points). This model should be used for selecting future transfer pairs.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1	–	9	9	7	9	9	13	7	13	13	13	13	13	7	12	13	7	7	9	13	2	13	3
2	23	–	40	25	35	56	53	33	55	40	27	39	53	34	39	51	24	38	46	44	18	47	34
3	9	20	–	4	17	36	45	10	47	31	13	25	44	10	30	31	13	16	26	22	7	37	9
4	21	41	41	–	20	35	41	38	41	38	28	36	41	39	32	41	24	35	32	41	28	41	15
5	15	30	26	12	–	32	40	13	35	31	27	21	40	25	28	37	26	20	30	40	26	33	22
6	12	33	38	15	23	–	45	15	40	36	34	27	43	31	25	35	21	27	37	41	18	36	15
7		5	16	4	9	24	–	5	20	9	15	9	34		7	18	8	9	10	22		17	
8	14	32	34	22	19	24	43	–	38	25	32	25	43	23	21	35	26	21	32	38	12	26	22
9	5	15	18	2	10	23	24	11	–	12	17	14	24	12	3	23	9	10	14	25	3	16	5
10	10	20	30	10	20	23	41	20	34	–	19	19	35	13	19	28	12	24	28	30	6	25	15
11	13	19	19	3	16	21	28	15	21	19	–	16	28	10	12	23	12	14	18	27	14	20	8
12	8	21	24	17	16	23	24	13	24	23	17	–	24	19	22	24	17	21	22	24	9	23	17
13	2	3	8		2	6	18	4	17	3		4	–	3	2	5		8	1	8	2	4	5
14	5	27	32	13	25	29	32	8	32	21	17	16	32	–	18	32	20	27	23	32	9	32	15
15	36	47	50	2	32	27	50	50	50	50	11	50	43	30	–	50	29	25	8	50	18	47	26
16	7	15	20	6	8	17	21	8	21	15	13	15	21	9	8	–	11	16	16	21	5	17	8
17	16	26	32	11	22	31	33	17	32	31	25	25	32	22	19	29	–	28	23	32	10	32	20
18	11	35	36	12	23	29	38	12	38	24	24	20	38	20	19	35	24	–	25	38	16	35	10
19	29	35	68		37	68	92	37	90	78	43	57	87	54	10	79	36	35	–	92	21	85	29
20	4	9	15	2	7	30	38	11	43	18	6	10	54	4	5	31	7	8	25	–	1	48	7
21	4	13	19	4	11	19	23	4	23	15	8	9	23	10	14	19	8	17	19	23	–	17	5
22	8	19	32	9	16	25	41	4	30	29	24	19	36	21	19	33	20	16	30	43	14	–	11
23	14	28	38	8	24	36	38	20	38	28	26	30	38	26	20	28	29	24	30	38	19	35	–

9 Toward a Metatheoretic Impossibility Result

The paradigm boundary identified in Sections 5 and 7 is an empirical finding: 20 attempts by Number Theory's algebraic tactics against Condensed Mathematics and Representation Theory produced 0 successes. But the uniformity of the failure mode—every attempt traces to a missing type-class instance—suggests that the barrier is not an engineering limitation but a *logical necessity*. Can this be proved?

9.1 The claim

> No tactic whose proof terms are built from the axioms of `CommSemiring` can close a goal whose type carries no `CommSemiring` instance.

This is a proof-theoretic statement: the derivable equalities of the algebraic fragment form a restricted class, and morphism-level equalities are provably outside it.

9.2 Three layers of formalization

Layer 1: Non-existence of instances. Prove in Lean that specific types lack specific instances:

```
-- Morphisms in an abstract category carry no CommSemiring
theorem no_CommSemiring_hom {C : Type*} [Category C]
    (X Y : C) (hne : X ≠ Y) :
    ¬ CommSemiring (X ⟶ Y) := ...
```

Combined with the documented precondition that `ring` requires `CommSemiring`, this yields: "`ring` cannot close goals of type $f = g$ where $f, g : X \longrightarrow Y$ in an abstract category." This is standard Lean mathematics—no metaprogramming reflection required.

Layer 2: Proof-term fragment analysis. Formalize *what proof terms `ring` can produce*. Mathlib's `Mathlib.Tactic.Ring` constructs proof terms via a ring normal form: compositions of `add_comm`, `mul_assoc`, `add_zero`, `mul_one`, `distrib`, and their consequences. Define this fragment as an inductive type:

```
inductive RingDerivable [CommSemiring R] :
    R → R → Prop where
  | refl : RingDerivable a a
  | add_comm : RingDerivable (a + b) (b + a)
  | mul_assoc : RingDerivable (a * b * c) (a * (b * c))
  | trans : RingDerivable a b → RingDerivable b c
      → RingDerivable a c
  ...
```

The impossibility follows from the type signature: `RingDerivable` has `[CommSemiring R]` as a parameter. Without the instance, the inductive type *cannot be instantiated*—in dependent type theory, the paradigm boundary is enforced by the type system itself. The theorem is: "the `ring` tactic produces a proof by constructing a `RingDerivable` witness and extracting $a = b$ via soundness; constructing the witness requires `CommSemiring`; therefore, if the goal type has no such instance, `ring` cannot produce a proof."

Layer 3: General paradigm boundary. The most ambitious version: for *any* tactic whose proof terms live in an axiomatic fragment F, and any goal type T that doesn't model F's axioms, the tactic cannot close the goal. This is analogous to separation results in proof complexity (Resolution vs. Cutting Planes vs. Frege systems). The algebraic fragment (`CommSemiring` axioms) and the categorical fragment (composition, identity, functoriality axioms) generate *non-overlapping* sets of derivable equalities—except where a type happens to model both (e.g., endomorphism rings).

9.3 Why this matters

A formal impossibility result would upgrade the paradigm boundary from an empirical observation ($n = 20$, $p = 0$) to a mathematical theorem. It would also establish that the tactic taxonomy of Section 4—structural combinators vs. algebraic procedures vs. domain-specific lemmas—is not merely a practical classification but reflects genuine proof-theoretic distinctions between the axiomatic fragments that different tactic classes exploit.

The deeper question is whether the barrier is *fundamentally* about type-class instances or about something more structural. Morphisms compose but do not add or multiply; composition is non-commutative and non-distributive; there is no zero morphism in a general category. The algebraic axioms that ring tactics exploit are *structurally incompatible* with the categorical axioms that morphism-level proofs use. Formalizing this incompatibility—not just for `ring` vs. morphisms, but for algebraic vs. categorical proof theory in general—would be a contribution to the foundations of formalized mathematics.

We plan to pursue Layers 1 and 2 as a companion paper to the Yanasse series, with the Lean formalization extending the existing formal verification system in the project's `Formal_Verification/` directory.

10 Conclusion

The initial Yanasse campaign—100 schema transfer attempts across 10 area pairs, producing 35 Lean-verified new proofs—demonstrates that a domain-independent relational analogy engine can systematically discover cross-domain connections between areas of formalized mathematics. The engine sees the "shape" of proofs without understanding their content; when that shape aligns with type-class compatibility, new proofs emerge.

The campaign's most important contribution is not the 35 new proofs themselves—each is a new proof of an existing theorem, not a new theorem in the mathematical sense—but the *structural map* of transferability across Mathlib. This map reveals:

1. A **two-factor model**: transfer success depends on source tactic profile $\times$ target type-class landscape.

2. An **element/morphism paradigm boundary**: algebraic decision procedures cannot cross from element-level to morphism-level proof domains, except where element-level sub-layers exist.

3. A **tactic taxonomy**: structural combinators transfer broadly (5+/7 targets); algebraic procedures transfer narrowly (1/3 targets, and only with type-class compatibility); meta-tactics transfer independently of QAP scores.

4. A **complementarity principle**: different source areas access non-overlapping layers of the same target, so multi-source campaigns achieve broader coverage.

These findings directly inform the Deep Vision theorem prover: prioritize Probability as a general-purpose analogy source; add type-class pre-filtering to the QAP matcher; weight analogy suggestions by the two-factor model; and maintain separate libraries for structural and algebraic tactic families.

The Yanasse method is not limited to the two source areas tested. The pipeline is fully automated (GPU-accelerated matching + AI-driven semantic adaptation + Lean compilation), and the structural map suggests that new source and target areas would each contribute unique transfer vectors.

The next ten pairs

The initial campaign processed the top 10 pairs from `area_pair_potential.parquet`, ranked by the naive model (sum of top-10 schema transfer gaps weighted by source-count). Section 8 introduced a transferability-weighted model that incorporates the campaign's empirical transfer rates per tactic class. The next 10 pairs are selected by the weighted model (Table 6), which reshuffles the rankings dramatically—Number Theory pairs drop out entirely, and three new source areas appear:

Rank	Pair	Pot.	Cand.	Prediction
1	Prob → Computability	68	19	High: structural combinators on decidability proofs
2	Prob → AlgTopology	68	20	High: structural combinators + shared topology
3	Prob → GroupTheory	57	16	High: structural combinators on algebraic targets
4	AlgGeom → Computability	56	30	Unknown: first test of AlgGeom as source
5	AlgGeom → Dynamics	55	30	Unknown: scheme-level tactics on mixed target
6	Prob → CategoryTheory	55	17	Moderate: structural tactics on categorical proofs
7	RepTheory → InfoTheory	54	18	Unknown: first test of RepTheory as source
8	Prob → LinearAlgebra	54	15	High: structural combinators + algebraic structure
9	AlgGeom → InfoTheory	53	33	Unknown: categorical + algebraic hybrid
10	AlgGeom → Condensed	53	33	Unknown: scheme-level tactics on categorical target

The weighted model makes several testable predictions. Probability → Computability (rank 1) and Probability → AlgebraicTopology (rank 2) should match or exceed Probability's 46% campaign average, since their candidates are dominated by structural combinators (`ext1`, `by_cases`, `any_goals`). Algebraic Geometry (ranks 4, 5, 9, 10) is the most interesting unknown: its tactic profile mixes categorical DSL-style tactics with scheme-level algebraic computation, and the campaign offers no prior data on how this hybrid transfers. Representation Theory → Information Theory (rank 7) tests whether a *target* area from the initial campaign can serve as a *source*—a role reversal that the two-factor model predicts should work if RepTheory's distinctive tactics (chain complex decomposition, module extensionality) have structural analogs in InfoTheory's proof architecture.

These 10 pairs are the next phase of the Yanasse project. Combined with the initial campaign, they will cover 4 source areas (Probability, Algebraic Geometry, Representation Theory, plus

continued Number Theory analysis), 9 target areas, and 200 schema transfer attempts—doubling the empirical base for the structural map of Mathlib transferability.

Acknowledgments

Named after Horacio Hideki Yanasse. The Deep Vision matching engine was originally developed for chess cognition research; its application to formal mathematics was enabled by the domain-independent design of `deep_vision_lib.py`. All computations were performed on a MacBook Air (Apple M-series, 32 GB unified memory) using PyTorch 2.8 on the MPS backend.

References

[1] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 1: Probability Theory → Representation Theory," ARGO LABS Report for Distribution, April 2026.

[2] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 2: Probability Theory → Set Theory," ARGO LABS Report for Distribution, April 2026.

[3] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 3: Probability Theory → Condensed Mathematics," ARGO LABS Report for Distribution, April 2026.

[4] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 4: Probability Theory → Field Theory," ARGO LABS Report for Distribution, April 2026.

[5] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 5: Probability Theory → Dynamics," ARGO LABS Report for Distribution, April 2026.

[6] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 6: Probability Theory → Model Theory," ARGO LABS Report for Distribution, April 2026.

[7] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 7: Probability Theory → Information Theory," ARGO LABS Report for Distribution, April 2026.

[8] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 8: Number Theory → Condensed Mathematics," ARGO LABS Report for Distribution, April 2026.

[9] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 9: Number Theory → Representation Theory," ARGO LABS Report for Distribution, April 2026.

[10] A. Linhares, "Yanasse: Finding new proofs from Deep Vision's analogies — Part 10: Number Theory → Dynamics," ARGO LABS Report for Distribution, April 2026.

[11] A. Linhares, "Deep Vision," ARGO LABS Internal Report, 2026.

[12] D. R. Hofstadter and the Fluid Analogies Research Group, *Fluid Concepts & Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. New York: Basic Books, 1995.

[13] D. R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. New York: Basic Books, 1999.

[14] D. R. Hofstadter and E. Sander, *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. New York: Basic Books, 2013.

[15] A. Linhares, "An active symbols theory of chess intuition," *Minds and Machines*, vol. 15, pp. 131–151, 2005.

[16] A. Linhares and P. Brum, "Understanding our understanding of strategic scenarios: What role do chunks play?" *Cognitive Science*, vol. 31, no. 6, pp. 989–1007, 2007.

[17] A. Linhares and A. E. T. A. Freitas, "Questioning Chase and Simon's (1973) 'Perception in Chess'," *New Ideas in Psychology*, vol. 28, no. 1, pp. 64–78, 2010.

[18] A. Linhares, A. E. T. A. Freitas, A. Mendes, and J. S. Silva, "Entanglement of perception and reasoning in the combinatorial game of chess," *Cognitive Systems Research*, vol. 13, no. 1, pp. 72–86, 2012.

[19] A. Linhares and D. M. Chada, "What is the nature of the mind's pattern-recognition process?" *New Ideas in Psychology*, vol. 31, no. 2, pp. 108–121, 2013.

[20] A. Linhares, "The emergence of choice: Decision-making and strategic thinking through analogies," *Information Sciences*, vol. 259, pp. 36–56, 2014.

[21] A. Linhares, "Deep Vision: Seeing the essence of proofs through analogy," ARGO LABS Internal Report, 2026.

[22] A. Linhares, "Deep Vision: A formal proof of Wolstenholme's theorem in Lean 4," ARGO LABS Report for Distribution, 2026.

[23] A. Linhares, "A sketch of Deep Vision's study of mathematics," ARGO LABS Report for Distribution, 2026.